POS TECH

HACKATON

A ONG Esperança Solidária atua há mais de 10 anos acolhendo crianças em situação de vulnerabilidade. Atualmente, a gestão de doadores e campanhas de arrecadação é feita de forma manual, limitando a capacidade da ONG de expandir sua ajuda. Para resolver esse problema, a diretoria decidiu criar a plataforma digital "Conexão Solidária", focada em escalabilidade, observabilidade e automação.

A missão dos alunos e alunas do curso é arquitetar e desenvolver o MVP dessa plataforma, entregando uma solução robusta e pronta para crescer.

Requisitos Funcionais Detalhados (O que deve ser feito)

1. **Autenticação e Autorização (RBAC)**
   - O sistema deve possuir autenticação baseada em Tokens JWT (JSON Web Tokens).
   - Devem existir dois perfis (Roles) distintos: GestorONG e Doador.
   - Endpoints de gestão devem ser bloqueados apenas para usuários com a role GestorONG.
2. **Gestão de Campanhas (Acesso: GestorONG)**
   - **Criar/Editar:** o sistema deve permitir o cadastro de campanhas contendo obrigatoriamente: Título (string), Descricao (string), DataInicio (datetime), DataFim (datetime), MetaFinanceira (decimal), e Status (Ativa, Concluida, Cancelada).
   - **Regra de Negócio:** uma campanha não pode ser criada com a data de término no passado e a meta financeira deve ser maior que zero.
3. **Cadastro de Doador (Acesso: Público)**
   - O sistema deve permitir o cadastro de novos usuários com os campos: Nome Completo, Email (deve ser único no banco), CPF (validar formato) e Senha (armazenada com hash, ex.: BCrypt).
4. **Painel de Transparência (Acesso: Público)**
   - **Endpoint de Listagem:** o sistema deve expor uma API pública que retorne apenas as campanhas com status Ativa.

- **Dados Retornados:** A listagem deve exibir Título, Meta Financeira e o **Valor Total Arrecadado** até o momento (calculado com base nas doações processadas).

5. **Processo de Doação (Acesso: Doador Logado)**
   - O doador logado deve poder enviar uma intenção de doação informando o IdCampanha e o ValorDoacao.
   - **Regra de Negócio:** a doação não pode ser feita para campanhas encerradas ou canceladas.

Requisitos Técnicos Obrigatórios (Como deve ser feito)

A solução não será avaliada apenas pelo funcionamento, mas pela arquitetura aplicada. É **obrigatório**:

- **Arquitetura de Microsserviços:**
  - A solução deve conter pelo menos dois microsserviços distintos (exemplo: uma API de Campanhas/Usuários e um Worker de Processamento de Doações).
- **Comunicação Assíncrona e Mensageria:**
  - Ao receber uma nova doação (Item 5), a API **NÃO** deve atualizar o valor arrecadado da campanha diretamente no banco de dados.
  - A API deve publicar um evento (ex.: DoacaoRecebidaEvent) em um broker de mensageria (RabbitMQ ou Kafka).
  - Um segundo serviço (Worker/Consumer) deve consumir essa fila e, então, atualizar o "Valor Total Arrecadado" da respectiva campanha.
- **Orquestração com Kubernetes:**
  - A aplicação deve rodar em um cluster Kubernetes (pode ser Minikube, Kind, Docker Desktop K8s, etc.). Devem ser entregues os arquivos .yaml (Deployments, Services, ConfigMaps).

- **Observabilidade (Zabbix e Grafana):**
  - A aplicação deve expor métricas de saúde (/health ou /metrics).
  - O Grafana deve possuir pelo menos um dashboard configurado exibindo métricas reais da aplicação rodando (ex.: Consumo de CPU/Memória dos pods ou contagem de requisições HTTP).
- **Pipeline de CI/CD Automatizado:**
  - O repositório deve conter um pipeline (GitHub Actions, Azure DevOps, etc.) que seja acionado a cada push na branch principal.
  - O pipeline deve compilar o código (.NET build) e gerar a imagem Docker. O deploy automatizado no Kubernetes é opcional, mas a geração da imagem no CI é obrigatória.

Requisitos Técnicos Opcionais (Bônus - Sem impacto na nota)

- Implementação de Testes de Unidade (xUnit/NUnit) no domínio da aplicação, rodando obrigatoriamente dentro da esteira de CI.
- Utilização de um API Gateway (ex.: Ocelot, KrakenD) roteando as requisições para os microsserviços.

Entregáveis Mínimos e Critérios de Aceite

1. **Repositório de Código Público**
   a. Obrigatório conter um arquivo README.md com instruções claras e em passo a passo de como subir a infraestrutura e a aplicação localmente para correção dos professores.
2. **Desenho da Arquitetura**
   a. Entrega de um diagrama claro mostrando os microsserviços, os bancos de dados, o broker de mensageria e as ferramentas de observabilidade.
   b. Documento em PDF justificando por que os bancos de dados X e Y foram escolhidos.
3. **Vídeo de Demonstração (Máximo de 15 Minutos):** o vídeo não deve focar em ler código linha a linha, mas sim em comprovar a arquitetura e o funcionamento. O roteiro do vídeo deve obrigatoriamente mostrar:

a. Explicação do Diagrama de Arquitetura.
b. Demonstração do Pipeline de CI executando e gerando a imagem Docker com sucesso.
c. Terminal mostrando os pods rodando no Kubernetes (kubectl get pods) e o dashboard do Grafana exibindo os dados em tempo real.
d. **Funcionamento:**
    i. Autenticação via Postman/Swagger e obtenção do token JWT.
    ii. Criação de uma campanha.
    iii. Simulação de uma Doação: mostrar o payload sendo enviado; em seguida abrir a interface do RabbitMQ/Kafka mostrando a mensagem passando pela fila e, por fim, consultar a API pública para provar que o valor da campanha foi atualizado pelo Worker.

4. **Relatório de entrega (PDF ou TXT) – esse arquivo deve ser postado na data da entrega, contendo:**
    - Nome do grupo.
    - Participantes e usernames no Discord.
    - Link da documentação.
    - Link do(s) repositório(s).
    - Link do vídeo salvo no Youtube ou lugar de sua preferência.

Lembramos que, caso você tenha qualquer dúvida, é só nos chamar no Discord!

POSTECH